

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:****Day:****Semester:****Lab 12: Model Deployment using Flask & Package Management**

Lab Title	CLO	BT	PLO	Weightage
Model Deployment using Flask REST APIs, Docker & uv				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

Objectives

- Convert an existing API from FastAPI to Flask
- Understand differences between API frameworks
- Deploy a trained model through a web interface and containerize it
- Utilize uv to manage modern Python environments and resolve version conflicts

Problem Statement

In the previous lab, you built a machine learning pipeline, tracked experiments using MLflow, serialized the trained model, and deployed it using a FastAPI-based REST API.

In real-world systems, different frameworks may be used depending on project requirements. Flask is a lightweight and widely used framework for building simple web applications and APIs. Furthermore, managing dependencies for complex ML packages (like MediaPipe for computer vision) often leads to version conflicts on newer Python releases.

In this lab, you will convert the FastAPI deployment into a Flask application. Following this, you will explore environment and version management using uv to resolve a simulated dependency conflict.

You are required to demonstrate the working system through screenshots of:

- The running Flask application and web interfaces
- Terminal outputs demonstrating dependency errors and their resolution

Part A: Understanding Existing API

Task 1: Analyze FastAPI Implementation

Review your Lab 11 FastAPI code and identify:

- How the model is loaded
- How input is received
- How predictions are returned

Answer:

- **Model Loading:** When the app starts it checks if `model.joblib` exists, and if it does it loads it using `joblib.load()`. Otherwise the model is set to `None` so the app knows its not available.
- **Input:** The `/predict` route takes a POST request with JSON data. It expects a key called "features" with a list of four numbers like `[age, income, hour, leak_feature]`.

- **Predictions:** Those features get put into a DataFrame and passed to `model.predict()`. The result (0 or 1) is sent back as a JSON response.

Relevant FastAPI code from Lab 11:

```
# model loading
mdl = joblib.load("model.joblib")

# predict endpoint
@app.post("/predict")
def predict(data: dict):
    features = data["features"]
    df = pd.DataFrame([features],
                      columns=["age", "income", "hour", "leak_feature"])
    result = mdl.predict(df)
    return {"prediction": int(result[0])}
```

Part B: Flask Setup

Task 2: Create Flask Application

Create a Flask application with:

- A home route (/)
- A prediction route (/predict)

Flask Application Code (flask_app.py):

```
import os
import joblib
import pandas as pd
from flask import Flask, request, render_template

app = Flask(__name__)

# load model
if os.path.exists("model.joblib"):
    mdl = joblib.load("model.joblib")
    print("model loaded ok")
else:
    mdl = None
    print("no model found")

cols = ["age", "income", "hour", "leak_feature"]
```

```
@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():
    if mdl is None:
        return render_template("index.html", error="model
                                not loaded")

    try:
        vals = [
            float(request.form["age"]),
            float(request.form["income"]),
            float(request.form["hour"]),
            float(request.form["leak_feature"]),
        ]

        df = pd.DataFrame([vals], columns= cols)
        result =mdl.predict(df)
        out= int(result[0])
        return render_template("index.html", prediction =
                                out)

    except Exception as e:
        return render_template("index.html", error= str(e)
                                )

if __name__ == "__main__":
    app.run(host= "0.0.0.0", port=5000, debug=True)
```

Part C: Model Integration

Task 3: Load and Use Model

Load the serialized model from Lab 11 and use it to make predictions.

Ensure:

- Correct input format
- Proper handling of numerical features

Answer:

The model gets loaded when the app starts using `joblib.load("model.joblib")` and stored in a variable called `mdl`. When someone submits the form, the four values (age,

income, hour, leak_feature) are taken from the form fields, converted to floats, and put into a list. This list is then turned into a DataFrame with the right column names so the model can understand it. After that `mdl.predict()` runs and gives back 0 or 1.

Model loading and prediction code:

```
# load model at startup
if os.path.exists("model.joblib"):
    mdl= joblib.load("model.joblib")

cols =["age", "income", "hour", "leak_feature"]

# inside predict route
vals = [
    float(request.form["age"]),
    float(request.form["income"]),
    float(request.form["hour"]),
    float(request.form["leak_feature"]),
]
df = pd.DataFrame([vals], columns= cols)
result =mdl.predict(df)
out= int(result[0])
```

Part D: Web Interface

Task 4: Create Input Form

Create a simple HTML form to take input features from the user.
Display prediction results on the webpage.

HTML Form Code (templates/index.html):

The form has four input fields for the features and a submit button. It posts to /predict and uses Jinja2 templating to show the result below the form when a prediction comes back.

```
<!DOCTYPE html>
<html>
<head><title>ML Prediction</title></head>
<body>
  <h1>ML Prediction</h1>
  <p>Lab 12 - Flask Model Deployment</p>
  <form method="POST" action="/predict">
    <label>Age</label>
    <input type="number" name="age"
      step="any" required><br><br>
    <label>Income</label>
    <input type="number" name="income"
      step="any" required><br><br>
    <label>Hour</label>
    <input type="number" name="hour"
      step="any" required><br><br>
    <label>Leak Feature</label>
    <input type="number" name="leak_feature"
      step="any" required><br><br>
    <button type="submit">Get Prediction</button>
  </form>

  {% if prediction is not none
    and prediction is defined %}
    <h2>Prediction: {{ prediction }}</h2>
  {% endif %}
</body>
</html>
```

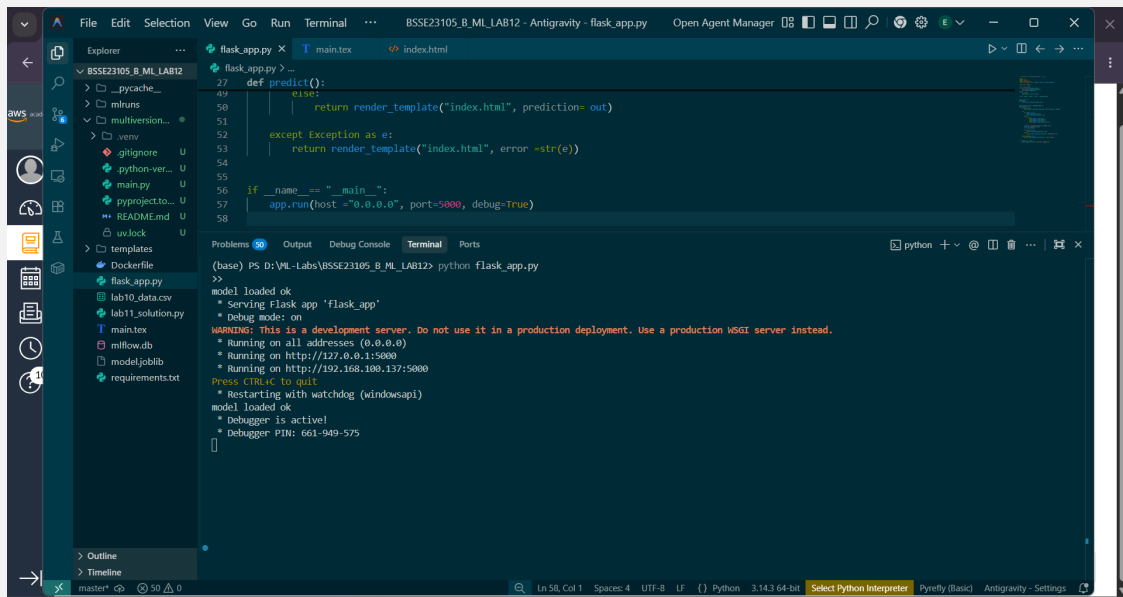
Part E: Testing and Screenshots

Task 5: Demonstration

Run the Flask application and capture screenshots of:

- Running server (terminal)
- Input form in browser
- Prediction output

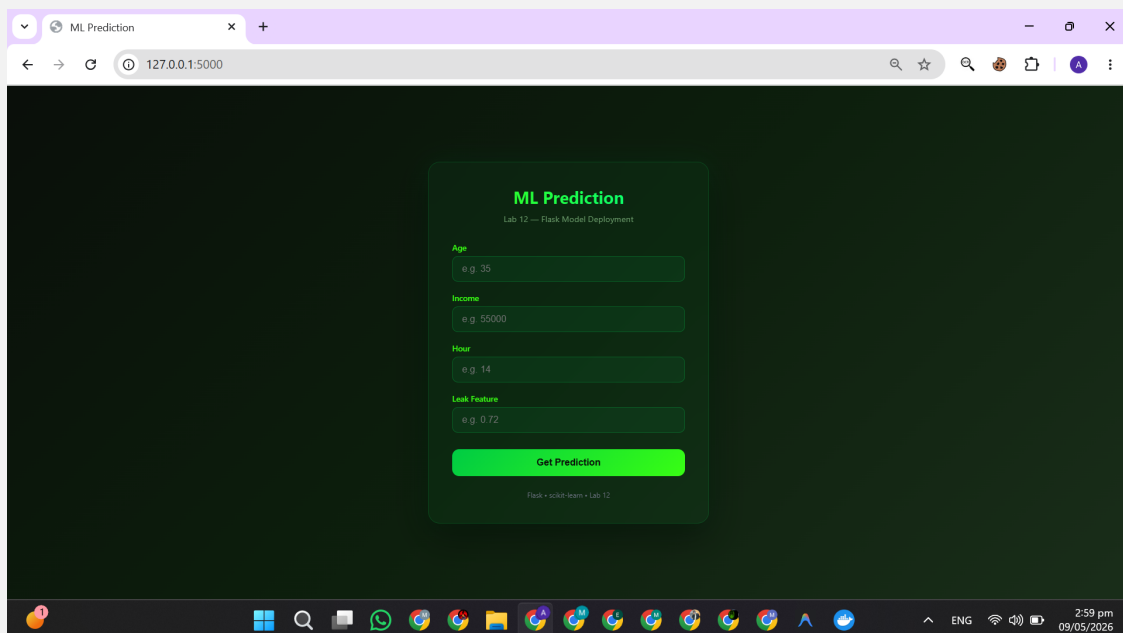
Screenshot – Flask server running in terminal:



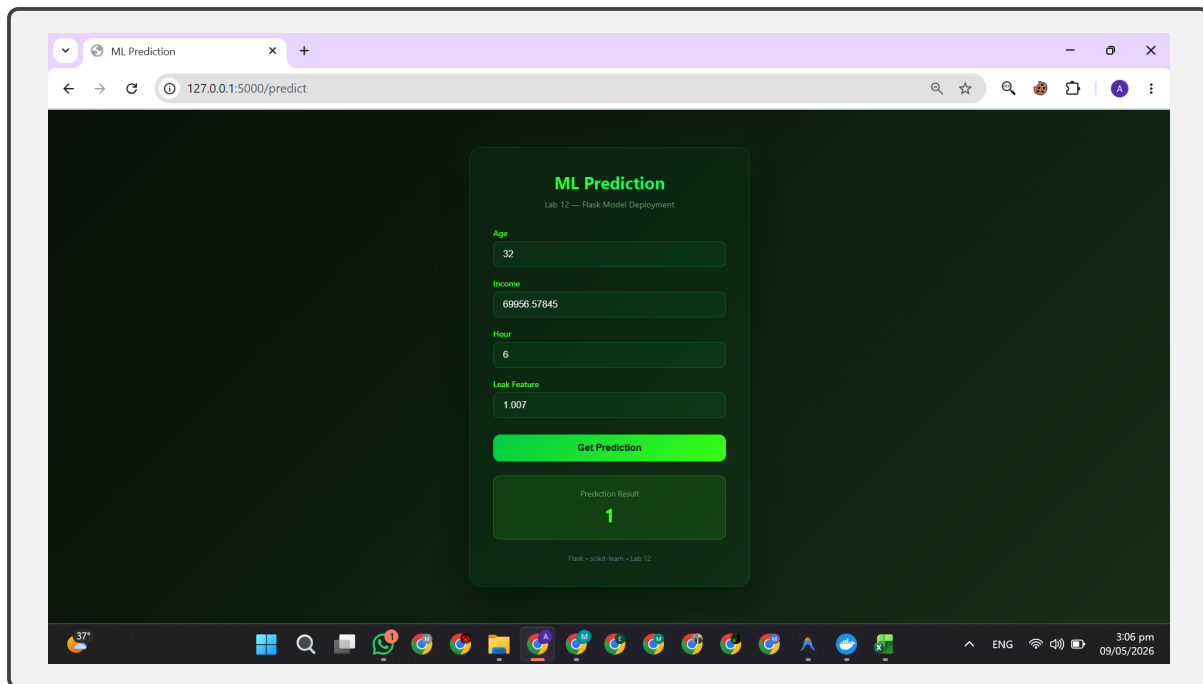
```
flask_app.py X  flask_app.py > ...
27 def predict():
49     else:
50         return render_template("index.html", prediction= out)
51
52 except Exception as e:
53     return render_template("index.html", error =str(e))
54
55 if __name__ == "__main__":
56     app.run(host ="0.0.0.0", port=5000, debug=True)
57
58

Problems Output Debug Console Terminal Ports
(base) PS D:\VL-Labs\BSSE23105_B_ML_LAB12> python flask_app.py
>>
model loaded ok
* Serving Flask app 'flask_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.100.137:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
model loaded ok
* Debugger is active
* Debugger PIN: 661-949-575
```

Screenshot – Input form in browser:



Screenshot – Prediction output:



Part F: Containerization using Docker

Task 6: Create Dockerfile

Create a Dockerfile for your Flask application.

Ensure:

- Base image uses Python
- Application dependencies are installed
- Flask app runs on port 5000

Dockerfile:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY flask_app.py .
COPY model.joblib .
COPY templates/ templates/

EXPOSE 5000
```

```
CMD ["python", "flask_app.py"]
```

requirements.txt:

```
flask>=3.0
pandas>=2.2
numpy>=1.26
scikit-learn>=1.5
joblib
```

Task 7: Build and Run Docker Image

Build the Docker image and run it locally.

Verify:

- Application runs inside container
- API is accessible via browser

Commands used:

```
docker build -t ml-flask-app .
docker run -p 5000:5000 ml-flask-app
```

Screenshot Required:

- Docker build output
- Running container and browser output

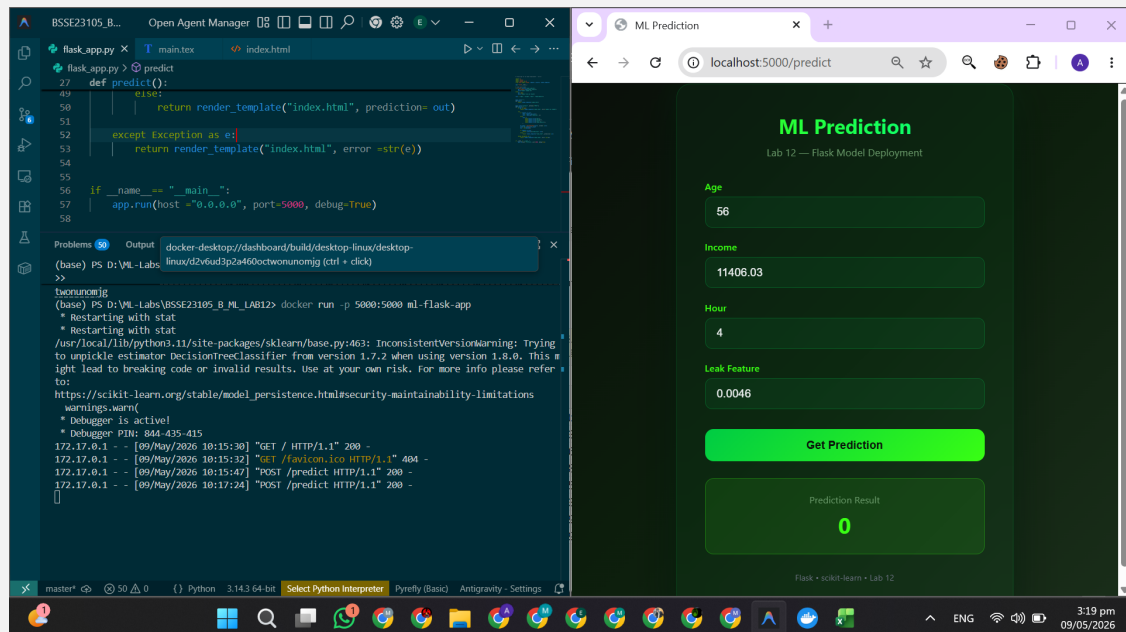
Screenshot – Docker build output:

```

(base) PS D:\VL-Labs\BSSE23105_B_ML_LAB12> docker build -t ml-flask-app .
>>
=> transferring context: 28B 0.0s
=> [1/7] FROM docker.io/library/python:3.11-slim@sha256:a5b427ace4980267d93db 0.1s
=> resolve docker.io/library/python:3.11-slim@sha256:a5b427ace4980267d93db 0.1s
=> [internal] load build context 0.1s
=> transferring context: 6.05kB 0.0s
=> CACHED [2/7] WORKDIR /app 0.0s
=> CACHED [3/7] COPY requirements.txt . 0.0s
=> CACHED [4/7] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [5/7] COPY flask_app.py . 0.1s
=> [6/7] COPY model.joblib . 0.1s
=> [7/7] COPY templates/ templates/ 0.1s
=> exporting to image 0.9s
=> exporting layers 0.2s
=> exporting manifest sha256:455f5dd1eadd2c7514361d358d3159854755f18f932 0.0s
=> exporting config sha256:95d0b6e0e9c9f5118b3a3f6c5d9a8e171c6eeafef4895 0.0s
=> exporting attestation manifest sha256:d0fbaec6f1255a1de3ee9548981a85eb 0.1s
=> exporting manifest list sha256:c452083dd368d31110ec1d5b8843c5f2658663 0.1s
=> naming to docker.io/library/ml-flask-app:latest 0.0s
=> unpacking to docker.io/library/ml-flask-app:latest 0.4s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/d2v6ud3p2a60cthuonnomjg
(base) PS D:\VL-Labs\BSSE23105_B_ML_LAB12>
```

Screenshot – Running container and browser output:



Part G: AWS ECR (Elastic Container Registry)

Task 8: Create ECR Repository

Create a repository in AWS ECR.

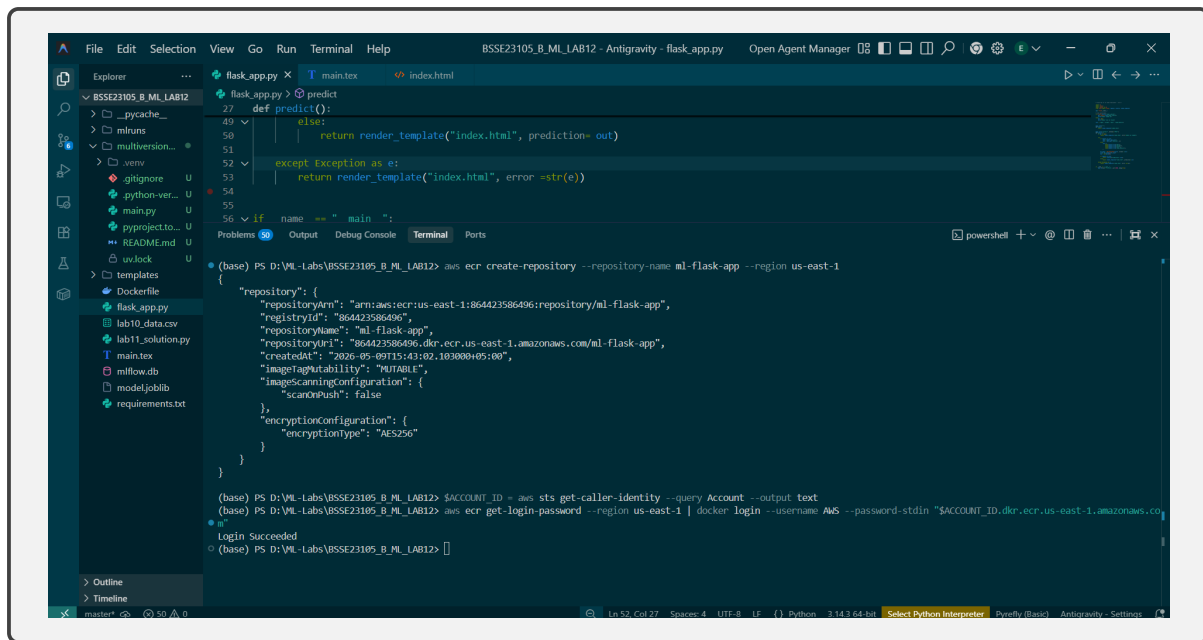
Login to ECR using AWS CLI and authenticate Docker.

Commands used:

```
aws ecr create-repository
--repository-name ml-flask-app
--region us-east-1

aws ecr get-login-password --region us-east-1 |
docker login --username AWS
--password-stdin
864423586496.dkr.ecr.us-east-1.amazonaws.com
```

Screenshot – ECR repository creation and Docker login:



Task 9: Push Docker Image to ECR

Tag and push your Docker image to ECR.

Commands used:

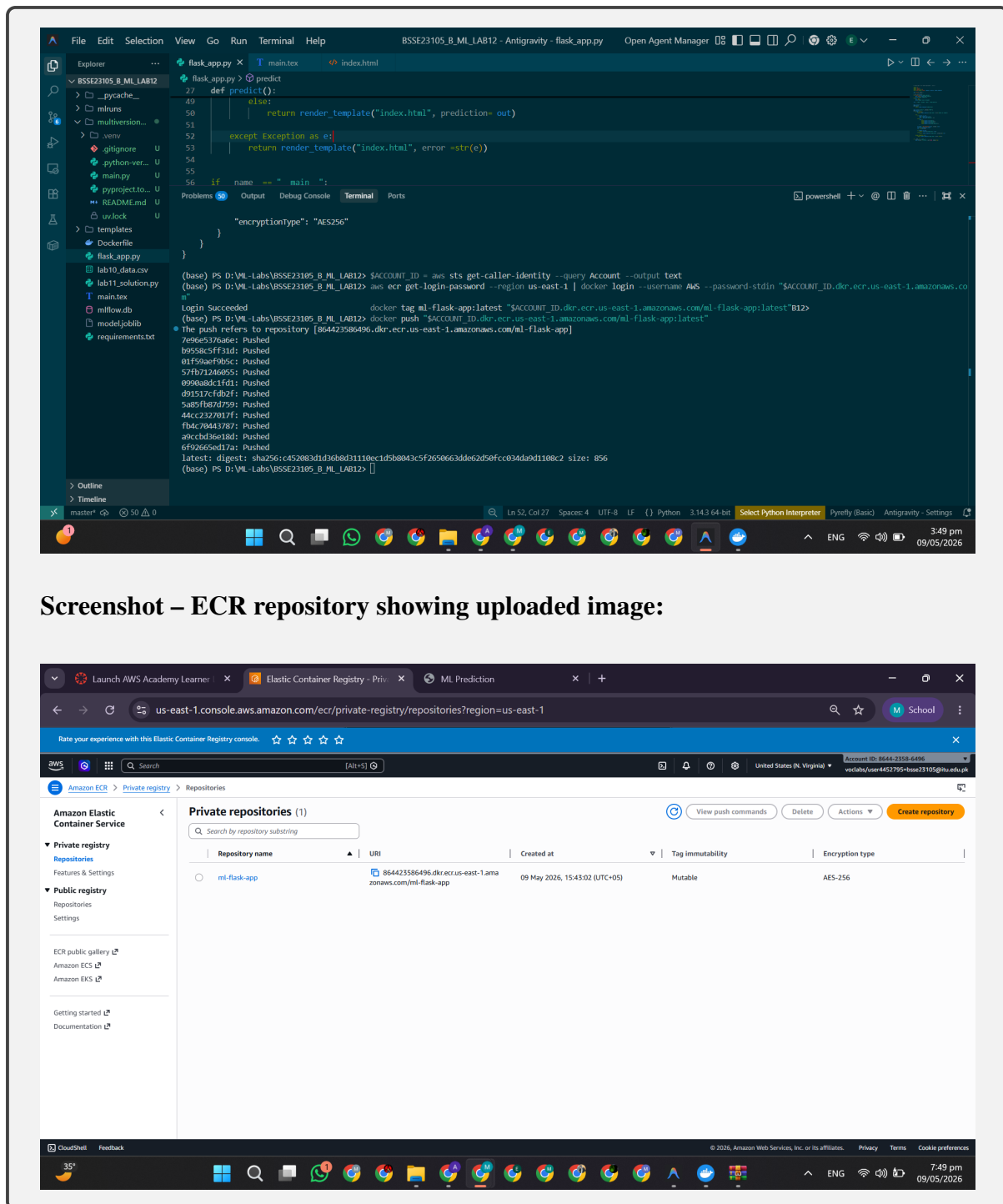
```
docker tag ml-flask-app:latest
864423586496.dkr.ecr.us-east-1.amazonaws.com/
ml-flask-app:latest

docker push
864423586496.dkr.ecr.us-east-1.amazonaws.com/
ml-flask-app:latest
```

Screenshot Required:

- Successful image push
- ECR repository showing uploaded image

Screenshot – Successful image push:



Part H: Deployment using Elastic Beanstalk

Task 10: Launch Environment

Using AWS Console:

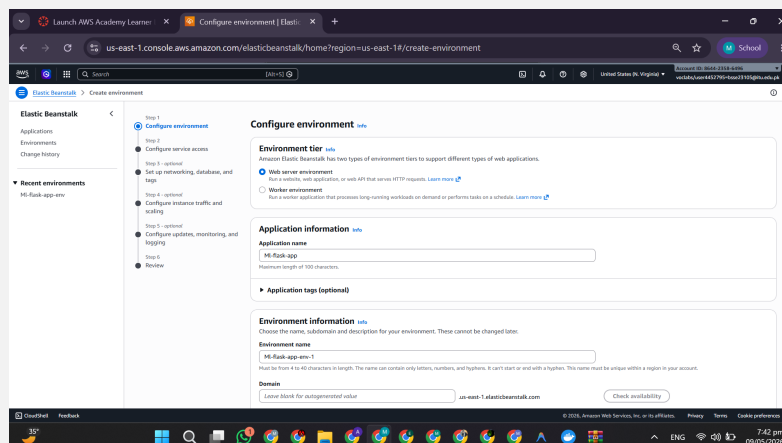
- Initialize Elastic Beanstalk application
- Configure environment (Docker platform)
- Deploy application using ECR image

Dockerrun.aws.json:

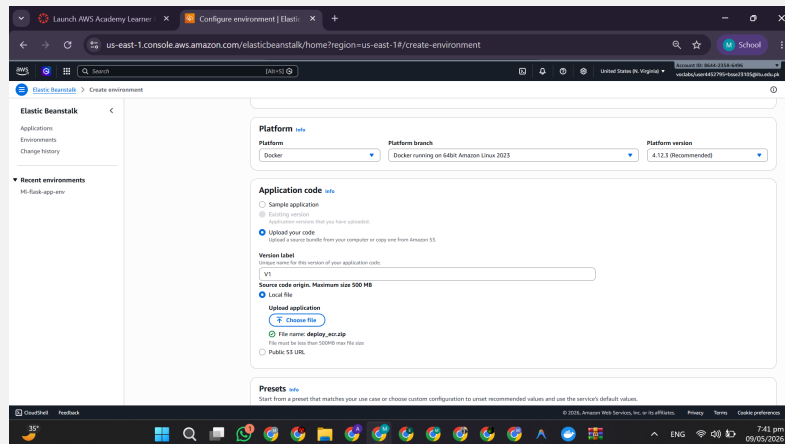
A Dockerrun.aws.json file was created to tell Elastic Beanstalk to pull the Docker image from ECR. This file was zipped and uploaded during environment creation.

```
{
  "AWSEBDockerrunVersion": "1",
  "Image": {
    "Name": "864423586496.dkr.ecr.us-east-1
      .amazonaws.com/ml-flask-app:latest",
    "Update": "true"
  },
  "Ports": [
    { "ContainerPort": 5000, "HostPort": 80 }
  ]
}
```

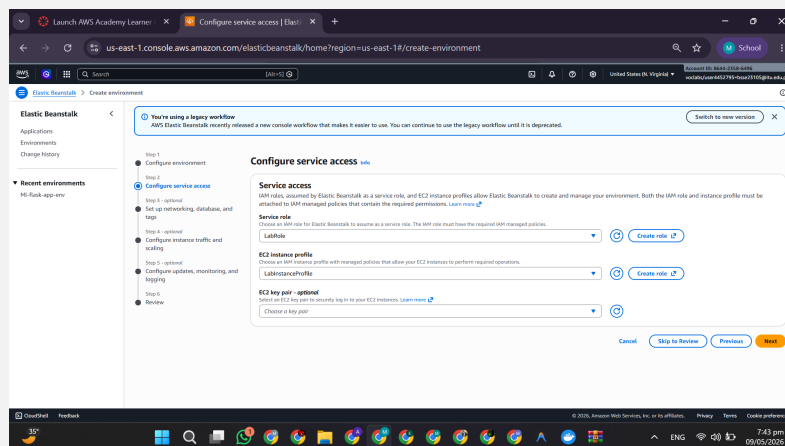
Screenshot – EB environment configuration:



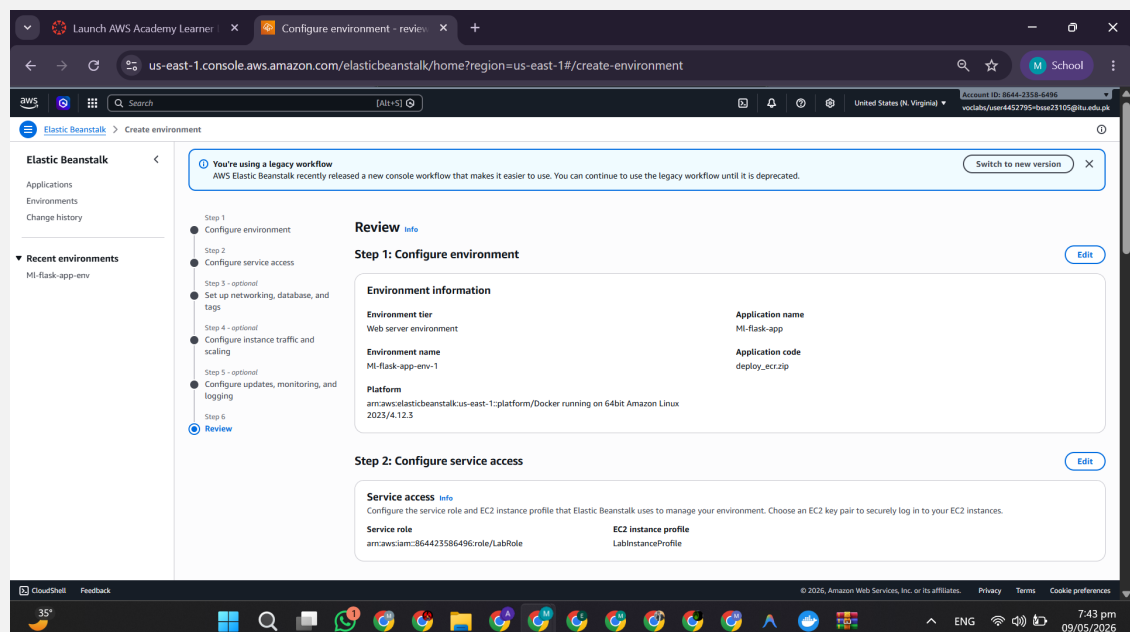
Screenshot – Platform selection and code upload:



Screenshot – Service access configuration:



Screenshot – EB environment launched:



Task 11: Verify Deployment

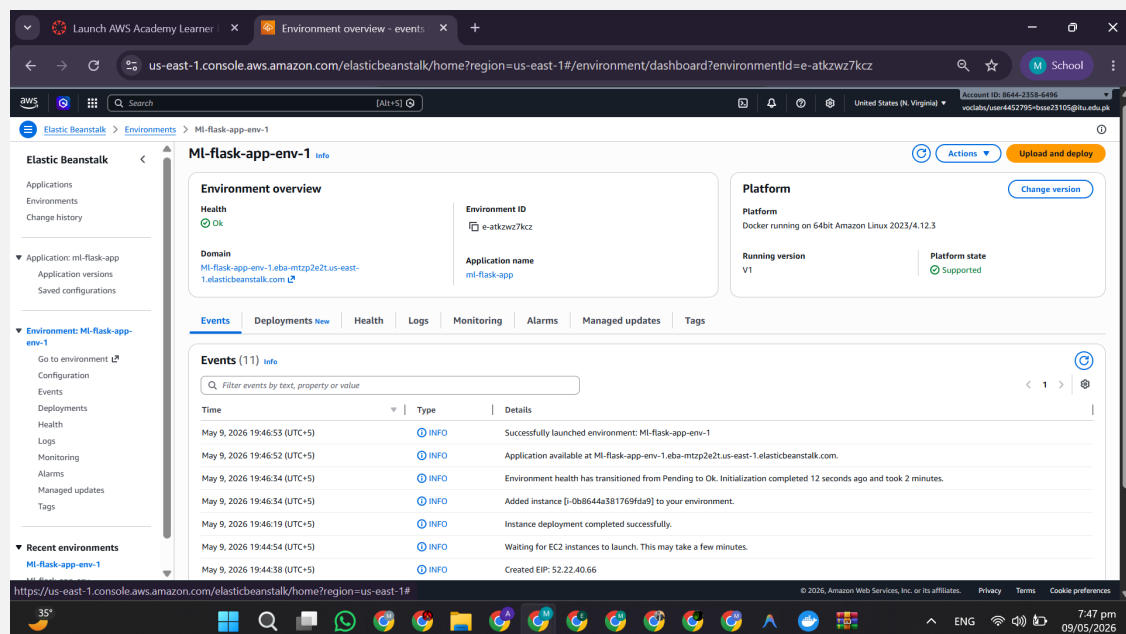
Access the deployed application via public URL.

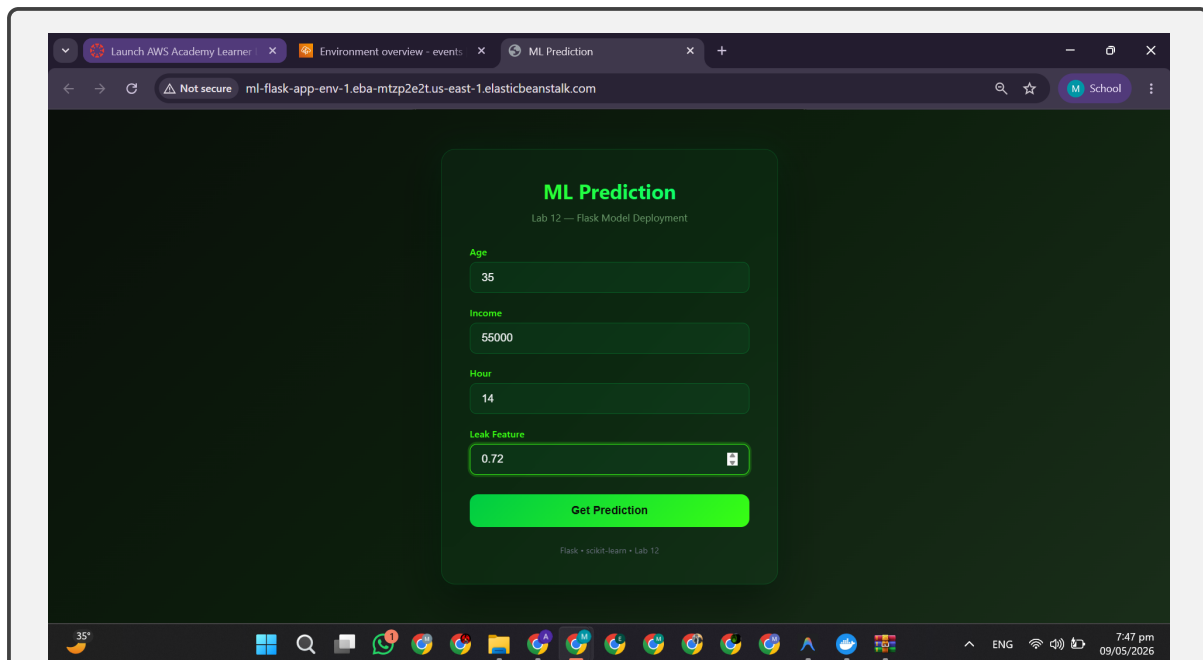
Verify:

- Flask app is running
- Predictions are working

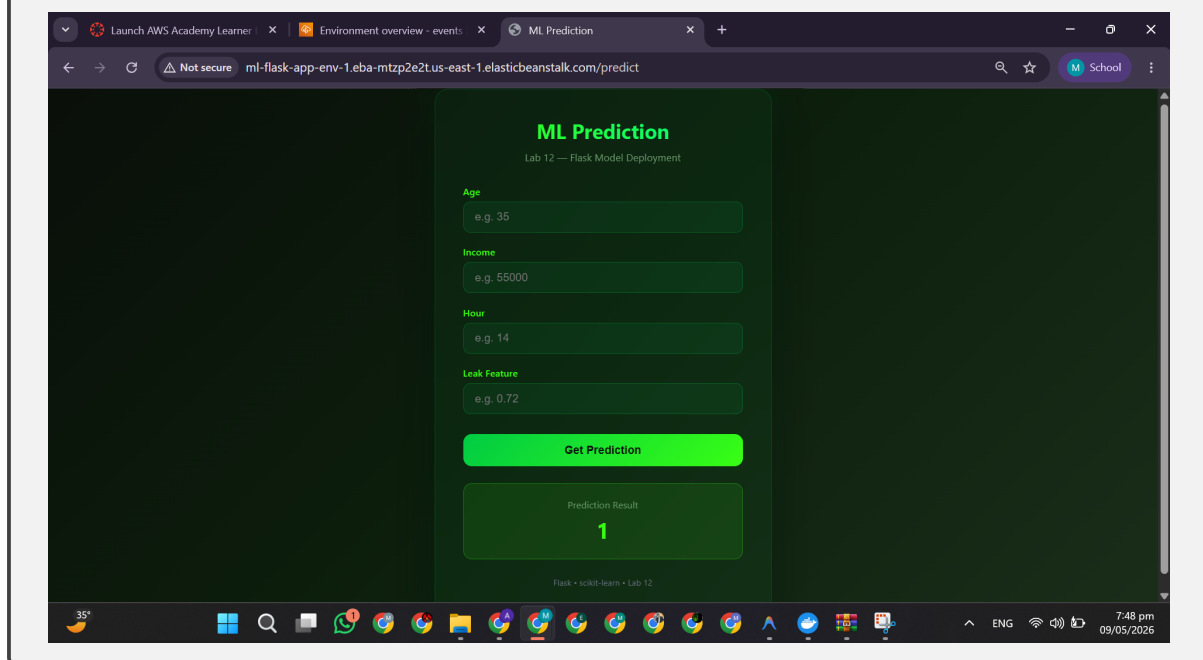
Screenshot Required:

- Elastic Beanstalk dashboard
- Live application URL in browser
- Prediction output

Screenshot – Elastic Beanstalk dashboard:**Screenshot – Live application URL in browser:**



Screenshot – Prediction output on deployed app:



Part I: Modern Package Management with uv [Bonus]

This section is a bonus task and was not attempted.

Part J: Reflection

Task 18: Reflection Questions

Answer the following:

- **What is the role of Docker in deployment?**

Docker puts the app and all its dependencies into one container. This way the app works the same on any machine, whether its your laptop, a test server, or the cloud. It removes the “works on my machine” issue.

- **Why use ECR instead of storing images locally?**

ECR stores Docker images on the cloud so anyone on the team can access them. It connects with other AWS services like Elastic Beanstalk and ECS. If you only store images locally, other people or servers cant use them.

- **What are the advantages of Elastic Beanstalk?**

EB handles the server setup, scaling and monitoring for you. You just upload your code or Docker image and it takes care of the rest. You dont have to manually set up EC2 instances or load balancers.

- **How does a tool like uv prevent “dependency hell” when working on multiple ML projects with different Python version requirements?**

uv keeps each project in its own isolated environment with its own lockfile. It can switch between different Python versions quickly and use environment markers to install packages only when the right Python version is being used. This way different projects dont mess up each others dependencies.